

Exekutiv sammanfattning

Dekodningen av base64-strängen ger byteföljden

b0 48 e8 0b 00 b0 69 e8 06 00 b0 21 e8 01 00 f4. Tolkningen av dessa bytes pekar kraftigt mot x86-maskinkod i 16-bitars läge, eftersom bl.a. B0 48 svarar mot instruktionen `mov al,0x48` (AL='H') och E8 iw är en **CALL**-instruktion enligt Intel/AMD-arkitekturen ¹ ². F4 motsvarar **HLT** ("halt") ³. Instruktionssekvenserna läser t.ex. som:

```
0000: B0 48      mov al,0x48      ; AL = 'H'
0002: E8 0B 00    call 0x0010      ; CALL relative +0x000B (bl.a. pushad
returnip)
0005: B0 69      mov al,0x69      ; AL = 'i'
0007: E8 06 00    call 0x0010      ; CALL relative +0x0006
000A: B0 21      mov al,0x21      ; AL = '!'
000C: E8 01 00    call 0x0010      ; CALL relative +0x0001
000F: F4         hlt             ; HALT (privilegerad instruktion)
```

Bytevärdena i hex och ASCII visas nedan (icke-tryckbara byten markerade som punkt):

```
Bytes (hex):  B0 48 E8 0B 00 B0 69 E8 06 00 B0 21 E8 01 00 F4
ASCII-analys: .H.. .i.. .!.. .
```

Här ses t.ex. tecknen 0x48='H', 0x69='i', 0x21='!'. Ingen sammanhängande text eller känd dataframställning framgår. Ordningsföljden av bytes tolkar man bäst som maskinkod. (Se AMD/Intel-dokumentationen för motsvarande opkoder ¹ ² ³.)

Möjliga instruktioner per arkitektur

- **x86, 16-bit (real mode)** – *Hög trolighet*. Vid 16-bitarsläge avkodas sekvensen korrekt som ovan. B0 ib är **mov AL,imm8** ¹ och E8 iw är **near CALL** med 16-bitars relativ offset ². Notera att CALL medför att returadressen (adressen efter CALL) läggs på stacken ⁴. Här pekar varje CALL (E8 xx 00) till adress 0x0010 (som ligger strax efter sekvensen) och trycker adressen för nästa instruktion på stacken. F4 är **HLT** ³ – ett systemhaltkommando som *bara* fungerar i privilegierat läge ⁵. Denna tolkning är koherent: koden innehåller flera `mov al` följt av CALL och avslutas med HLT, vilket är ovanligt men möjligt som en liten 16-bitars kodsnuitt.
- **x86, 32/64-bit** – *Låg trolighet*. I 32/64-bitarsläge förväntas CALL använda 4-bytes offset, vilket inte stämmer med de 2-bytes som ges här. Försök att disassemblera som 32-bitars resulterar i ofullständiga instruktioner (t.ex. E8 0B 00 B0 69 skulle behöva fyra offset-bitar). Endast `mov al,imm8`-delen (B0 xx) skulle fortfarande avkodas, men CALL-instruktionerna blir felaktiga eller ofullständiga. Därför matchar inte dessa bytes en 32- eller 64-bitarsinstruktionsström i IA-32/AMD64-läge.

- **ARM (32-bit) eller MIPS** – *Mycket låg trolighet*. I ARM- och MIPS-instruktionsset finns inga enkla instruktioner som motsvarar byteföljden. ARM-instruktioner är 4 bytes långa och har andra kodmönster (t.ex. kodbitar för "cond" fält). Våra hex-ord (big-/little-endian) matchar ingen vanlig ARM- eller MIPS-opcode (t.ex. `0xB048E80B` eller `0x0BE848B0` har inget uppenbart betydelsefullt mönster). Ingen referens i ARM-arkitekturmanualer (v7/v8) visar att t.ex. `B048E80B` skulle vara giltig. Så detta ser inte ut att vara ARM- eller MIPS-kod.
- **Data/strukturer eller text** – *Mycket låg trolighet*. Sekvensen innehåller inga tryckbara textsträngar utöver spridda ASCII-tecken ("H", "i", "!") och flera nollor. Det finns ingen uppenbar datastruktur (t.ex. pekare eller bilddata). Ingen känd formaterad data (t.ex. UTF-8-text, 32-bitars heltal på rimliga värden osv.) framträder. Byteföljden är heller inte slumpmässig nog för att vara t.ex. krypterade data (det finns upprepade mönster med `B0` och `E8`). Den upprepade strukturen visar snarare instruktioner (se ovan).

Instruktionssekvenser (exempel)

```
; x86 16-bit (real mode) disassembly
0000: B0 48      mov al, 0x48      ; AL := 'H'
0002: E8 0B 00      call 0x0010      ; nära CALL, push IP=0005
0005: B0 69      mov al, 0x69      ; AL := 'i'
0007: E8 06 00      call 0x0010      ; nära CALL, push IP=000A
000A: B0 21      mov al, 0x21      ; AL := '!'
000C: E8 01 00      call 0x0010      ; nära CALL, push IP=000F
000F: F4          hlt              ; HALT (privileged instruktion)
```

- I 32-/64-bitarsläge skulle `E8` instruera att läsa fyra bytes offset, vilket inte stämmer här. Ett exempel på hur en 32-bitarsdisassembler kan tolka den, men som blir felaktigt, är:

```
; x86 32-bit (försök - ogiltiga CALL-offsets)
0000: B0 48      mov al,0x48
0002: E8 0B 00 B0 call 0x69B0000F ; (ogiltig: CALL med 32-
bitaroffset)
0007: 69 E8 06 00
; ... fortsätter inte korrekt
```

Det blir uppenbart att sekvensen inte är avsedd som ren 32-bitars kod, eftersom CALL-instruktionerna inte är fullständiga.

- ARM- eller MIPS-disassembler skulle kräva omgruppering i 4-byte ord (t.ex. `0x0BE848B0`), men inga kända ARM/MIPS-mönster framkommer. Till exempel ger `0x0BE848B0` (ARM litte-endian) ingen match i ARM-v7 manualen. Därför avfärdas dessa arkitekturer.

Tolkning: Den mest troliga tolkningen är att detta är en kort bit x86-maskinkod (möjligen en shellkod eller demo-snutt i 16-bitars real mode). Mönstret med `mov al,<char>` följt av `call` (som skjuter returadr på stacken ⁴) och ett avslutande `hlt` är typiskt för en form av "position-independent" teknik eller ett specialfyllnadssegment. Notera att `HLT` är privilegierat ⁵, vilket betyder att i en typisk skyddad eller användarläge-omgivning (Linux/Windows) skulle detta snarare ge undantag. En möjlig tillämpning är i en 16-bitars omgivning (t.ex. BIOS eller DOS), där `hlt` faktiskt stannar CPU: n. Utan

ytterligare kontext är det inte möjligt att avgöra koden av en fullständigt fungerande exploit eller payload.

Sökning i signaturdatabaser

En sökning efter denna byteföljd i offentliga datakällor (t.ex. Exploit-DB, shellcode-databaser, virus-signaturarkiv) gav inga tydliga träffar. Ingen existerande dokumenterad exploit eller shellkod visas matcha exakt denna sekvens. Det innebär att det sannolikt är en skräddarsydd eller ovanlig kodbit, eller möjligen provkod/demo. Utan träffar kan vi inte bekräfta en specifik välkänd trojan/shellcodetechnologi.

Statisk analys

- **Instruktionsflöde:** Eftersom alla instruktioner är linjära utan grenar (utöver de korta CALLs) är kontrollflödet triviale. Varje CALL går till adressen just utanför kodsekvensen, vilket trycker returadressen på stacken ⁴. Det finns inga hopp eller loopar att analysera.
- **Systemanrop / hårdvaruåtgärder:** Ingen av instruktionerna anropar operativsystemet (ingen `int 0x80` eller `syscall`). Det enda händelseframkallande är `HLT` som i praktiken stoppar processorn (inom x86). Inga I/O-instruktioner (som `INT 21h` i DOS) syns.
- **Strings / ASCII:** De enda ASCII-tecken som dyker upp är `H`, `i`, `!` ("Hi!"), men de är inte i följd och omväxlande med kod, vilket talar emot att det är en enkel text. Inga andra läsbara strängar finns.
- **Bytefrekvens:** Bytesekvensen innehåller upprepningar (`B0` före varje konstant och `E8` före varje offset), vilket är typiskt för instruktioner snarare än slumpmässig eller krypterad data.
- **Kod vs data:** Givet de tydliga instruktionerna stöder analysen att detta är kod.

För att inte exekvera potentiellt skadlig kod kan man följa vanliga råd för *offensiv analys*: först statisk desassemblering (t.ex. i IDA eller Ghidra) för att studera strukturen ⁶. Kontrollera instruktionernas innebörd med officiella källor (som Intel/AMD-manualer) samt kända teknikrapporter. Observera att källkoden trycks halt (privilegium) vilket är ovanligt för användarkods-shellkod.

Dynamisk analys (säker körning)

Eftersom denna kod inkluderar `HLT` (som stoppar CPU) **bör den inte köras direkt på ett vanligt system** utan i en säker miljö. Rekommenderade metoder är att använda en emulator eller virtuell maskin:

- **Bochs:** En fullständig x86-emulator ⁷ som stegvis exekverar varje instruktion, säkert men långsamt. Bochs kan köra i 16-bitars real mode, vilket gör det lämpligt för denna kod.
- **QEMU:** Snabbare emulering med dynamisk översättning ⁸, men man bör justera till 16-bitarsläge.
- **Virtuell maskin:** Även VMware/VirtualBox kan användas, men då i en gäst-OS som stöder real mode (t.ex. en DOS-låda). Fördelen med Bochs/QEMU är hög kontroll och att man kan single-steppa och inspektera stack/minne utan risk.
- **Sandbox:** Program som Sandboxie eller Cuckoo kan hantera koden om den integreras i ett större malware-exempel, men för enstaka instruktioner är Bochs/QEMU att föredra ⁷.

Dynamiskt skulle man i en emulator t.ex. avbryta körning innan `HLT` inträffar, eller sätta CPU i virtuellt ring-0-läge. Detta kan avslöja exakt vad CALL-instruktionerna gör med stacken och eventuellt avslöja

om det finns kod eller data direkt efter sekvensen (även om vår kod slutar precis vid HLT). En möjlig experimentell analys är att logga stackinnehållet efter varje CALL. Eftersom exekvering av dessa instruktioner i verklig hårdvara skulle hänga datorn, är emulering nödvändig.

Jämförelse av tolkningar

Tolkning	Trovärdighet	Viktiga bevis	Nästa steg / åtgärd
x86 16-bit-kod	Hög	Kända opkoder: <code>B0 imm8</code> (mov AL) ¹ och <code>E8 iw</code> (CALL) ² . Mönstret överensstämmer med kod.	Fortsätt statisk analysera i disassembler. Emulera i real mode (t.ex. Bochs) och observera stack/pop.
Shellkod för DOS/BIOS	Medel	Innehåller tecken "H", "I", "!" i AL, och HLT (kan signalera programslut i real mode). Inga systemanrop, privilegierad HLT.	Om målplattformen är DOS/Bios, testa körning i VM, kontrollera om adress på stack avspeglar data/sträng.
x86 32/64-bit-kod	Låg	CALL-instruktioner (E8) kräver 4-byte offset i 32/64-bit, vilket sekvensen saknar.	Avfärda eller omöjlig i sin helhet. Ingen vidare åtgärd för x86-32/64.
ARM/MIPS-maskinkod	Mycket låg	Ingen rimlig avkodning; byte-mönster matchar inte ARM/MIPS-opkoder.	Avfärda. Behöver ej analyseras vidare.
Data/strukturer	Mycket låg	Inga läsbara strängar eller tydlig datakonfiguration. Upprepad struktur liknar instruktioner.	Inget indikerar data. Förmodligen ej datasekvens.

Analysens arbetsflöde

flowchart LR

```

A[Utgång: Base64-sträng] --> B[ByteSekvens i hex/ASCII]
B --> C{Gissade arkitekturer}
C --> D[x86 16-bit]
C --> E[x86 32/64-bit]
C --> F[ARM/MIPS]
D --> G[Disassembler (x86-16)]
E --> H[Disassembler (x86-32/64)]
F --> I[Disassembler (ARM/MIPS)]
G --> J[Instruktionslista]
H --> K[Ofullständig / ogiltig]
I --> L[Ofullständig / ogiltig]
J --> M[Identifiera instruktionernas betydelse]
M --> N[Matchning mot signaturdatabaser]
N --> O[Statisk analys: kontrollflöde, strängar]

```

```
O --> P[Beslut: Tolkningar]
P --> Q[Dynamisk analys/Emulering]
```

Figur: Arbetsflöde för analys – från dekodning via disassemblering till tolkningar och vidare analyssteg.

Antaganden och begränsningar

- **CPU-läge:** Vi förutsätter 16-bitars real mode för bästa matchning. I annat läge (protected/long mode) skulle HLT vara otillgängligt och CALL-storlekar annorlunda.
- **Endianness:** x86 är little-endian, vilket vi använt vid disassembl(er)ingen. Inga indikationer föreslår annat byte-ordning.
- **Omgivningsekontext:** Vi antar att koden antingen är fristående eller del av en större payload. Ingen information om upphämtningsadresser (t.ex. segmentregistrens värden) finns, vilket kan påverka exakta adresstolkningar.
- **Uppgiftshanterare/shell:** Ingen explicit system-/shellanrop hittas, så vi antar att koden är low-level (t.ex. BIOS) eller förbereder data/instruktioner istället för att utföra kända systemcalls.

Detta är en statisk analys baserad på givna bytes och kända instruktioner ¹ ² ³. Om denna kod används i ett verkligt sammanhang kan andra faktorer (som segmentgränser, efterföljande data eller omgivande instruktioner) ge ytterligare insikt. Utan sådan kontext kvarstår osäkerheter.

Slutsats

Byteföljden `b0 48 e8 0b 00 b0 69 e8 06 00 b0 21 e8 01 00 f4` verkar vara *x86-maskinkod i 16-bitarsläge*, inte text eller vanliga data. Den innehåller instruktionerna `mov al,<imm8>`, `call <rel>` och `hlt` ¹ ² ³. Den exakta funktionen är oklar – möjlig demonstration av att ladda registren med tecken och använda CALL för att få en adress på stacken. På grund av HLTs privilegium och avsaknad av systemanrop är detta inte typisk användarläges-shellkod i moderna OS. Vidare testning bör ske i en emulator (t.ex. Bochs eller QEMU) med 16-bitars kontext för att studera stackinnehåll och eventuella efterföljande data utan att slå ut värddatorn ⁷.

TL;DR: Dekodningen av base64 ger bytes som motsvarar x86 16-bitarsinstruktioner (två MOV och CALL-följder, avslutande med HLT) ¹ ². Ingen text eller känd data hittas. Troligen är det ett litet x86-kodfragment (möjligen shellkod) i real mode. Vidare analys bör ske statiskt i disassembler och dynamiskt i säker emulator (t.ex. Bochs/QEMU) ⁷.

¹ Untitled Document

https://kib.kiev.ua/x86docs/AMD/Optimization/22007E_AMD%20Athlon%20Processor%20x86%20Code%20Optimization%20Guide.pdf

² AMD64 Architecture Programmer's Manual, Volume 3, General-Purpose and System Instructions

http://bluewaters.ncsa.illinois.edu/liferay-content/document-library/amd_3.pdf

³ ⁵ HLT — Halt

<https://www.felixcloutier.com/x86/hlt>

⁴ assembly - Why would a program contain a call instruction targetting the address immediately following that instruction? - Reverse Engineering Stack Exchange

<https://reverseengineering.stackexchange.com/questions/1654/why-would-a-program-contain-a-call-instruction-targetting-the-address-immediate>

6 Statically Reverse Engineering Shellcode Techniques: Stage 1 | Offset Training Solutions

<https://www.Offset.net/reverse-engineering/common-shellcode-techniques/>

7 8 virtual machines - How can I analyze a potentially harmful binary safely? - Reverse Engineering Stack Exchange

<https://reverseengineering.stackexchange.com/questions/23/how-can-i-analyze-a-potentially-harmful-binary-safely>